

Projet Freescord

Lunzitisa Luila

4 mai 2026

1 Introduction

Le projet Freescord est une application client-serveur en C permettant à plusieurs utilisateurs de communiquer via un serveur central. Il met en œuvre des concepts de programmation réseau, gestion de concurrence et manipulation de buffers.

2 Architecture

Le projet est composé de :

- serveur.c : gestion des clients et diffusion des messages
- client.c : communication utilisateur
- buffer.c / buffer.h : gestion des entrées/sorties
- user.h : structure utilisateur
- utils.h : changer des lignes
- list.h : liste des utilisateurs

3 Modifications

3.1 Client

- Organisation du main avec :

```
1 int envoyer(...);
2 void bienvenue(...);
3 void negocier_pseudo(...);
```

- Utilisation de `snprintf()` pour éviter les débordements

3.2 Buffer

```
1 int pos_curseur;
2 int taille_buffer;
3 int has_ungetc;
4 int eof;
5 char ungetc_char;
```

Ajout de la fonction `remplir_buff()`.

3.3 Serveur

- Limitation : restreindre le nombre de client

```
1 #define NB_CLIENT 10
```

- Mutex : protéger les zones critiques lors du parcours de la liste des utilisateurs, l'ajout d'un client connecté, la suppression d'un client déconnecté dans la liste ou si le serveur est complet

```
1 pthread_mutex_t users_mutex;
```

- Comparaison avec `strcasecmp()` notamment pour vérifier les mots interdits en minuscules ou majuscules.

```
1 #include <strings.h>
```

3.4 User

- Identification : permet de repérer l'utilisateur

```
1 char *nickname;
```

4 Problèmes rencontrés

- Limitation et vérification des pseudos difficiles
- Gestion des lignes partielles
- Boucle infinie avec Ctrl+D (pseudo trop long)

5 Solutions

5.1 Déconnexion

```
1 int deco;
```

Si le client se déconnecte lors de la demande de pseudonyme, on sort directement de la liste des utilisateurs connectés donc du serveur.

5.2 Lignes partielles

```
1 if(!l && buff_ready(b)) {
2     poll(fds, 1, -1);
3     continue;
4 }
```

5.3 Ctrl+D

```
1 if(fgets(pseudo, sizeof(pseudo), stdin) == NULL)
2 {
3     printf("Vous avez quitte le Freescord!\n");
4     close(sock);
5     buff_free(b);
6     exit(0);
7 }
```

6 Conclusion

Ce projet m'a permis de mettre en pratique les concepts fondamentaux de la programmation système sous Linux :

- La programmation réseau avec les sockets TCP.
- Le multiplexage d'entrées/sorties avec `poll()`.
- La lecture bufferisée pour éviter les appels système excessifs.
- La gestion de la concurrence avec les threads POSIX et les mutex.
- La communication inter-threads via un tube.
- L'implémentation d'un protocole applicatif (CRLF, négociation de pseudonyme).

Les principales difficultés rencontrées concernaient la gestion correcte du buffer (priorité des opérateurs, distinction entre données disponibles et ligne complète) et la synchronisation entre les threads du serveur.